Passenger Van Rental System

BACKGROUND

Passenger van rental services provide transportation for families, groups, businesses, and other customers who need a vehicle for a specific period. A rental operation must keep track of its vans, customer information, rental schedules, return dates, fees, and vehicle maintenance. Staff also need to know which vans are available, currently rented, or temporarily unavailable because they are undergoing maintenance.

When these activities are managed through separate paper records or informal tracking methods, it can become difficult to maintain accurate and consistent information. Staff may have difficulty confirming vehicle availability, checking rental due dates, calculating late fees, preserving customer and rental history, or identifying vehicles that should not be rented. Missing or inconsistent records can cause delays, confusion, and avoidable errors in daily operations.

The Passenger Van Rental System is a Java-based, menu-driven application intended to organize these tasks in one program. Based on the provided source code, it manages vans, customers, rental transactions, rental staff, and maintenance records. It also supports overdue-rental checking, simulated email receipts and SMS notifications, input validation, and saving data to text files so records can be loaded again when the program is restarted.

PROBLEM STATEMENT

The rental operation needs a consistent way to manage vehicle and rental information. The system addresses the following operational problems:

1. Vehicle availability may be difficult to confirm without a single, organized record of available, rented, and maintenance-status vans.
2. Customer details, proof-of-identity information, and rental history need to be stored and retrieved consistently.
3. Rental start dates, due dates, returns, and late fees require accurate tracking to reduce manual calculation errors.
4. Vehicles under maintenance must be identified and prevented from being rented until maintenance is completed.
5. Staff need a practical way to identify due or overdue rentals and send customer reminders.
6. Records need to remain available after the application closes, rather than being lost at the end of a session.

OBJECTIVES

General Objective

To develop a Java-based Passenger Van Rental System that organizes van, customer, rental, staff, maintenance records, supports rental and return transactions, and preserves operational data through file-based storage using object-oriented programming principles.

Specific Objectives

1. To create and manage customer records, including contact details, address, and valid identification information.
2. To add, view, search, and manage standard and luxury vans, including their availability and maintenance status.
3. To record van rentals and returns, calculate rental fees, and determine applicable late fees for overdue returns.
4. To maintain rental records that identify the customer, rented van, dates, rental status, and staff involved in the transaction.
5. To record maintenance activity and prevent a van under maintenance from being rented.
6. To check due and overdue rentals and provide simulated email receipts and SMS expiration reminders.
7. To apply encapsulation, inheritance, polymorphism, abstraction, object relationships, collections, exception handling, and file persistence where appropriate.
8. To validate user input and save and retrieve system records through text files between program sessions.

TARGET USERS

Rental Staff

Rental staff are the primary users of the application. The program does not currently require a login. Staff identify themselves by entering an existing staff ID when carrying out actions such as renting or returning a van and recording maintenance. They can manage customer records, manage vans, process rentals and returns, review rental information, and view maintenance history.

Customers / Renters

Customers are the people whose information and rental transactions are recorded by staff. In the current implementation, customers do not directly operate the program. Their records include contact information, address, and identification details needed by the rental process. They may receive a simulated email receipt or SMS reminder through the application.

Rental Business Owner or Supervisor

A business owner or supervisor may use the system's lists and records to review vehicle status, rental activity, overdue transactions, and maintenance history. The source code does not implement a separate supervisor account or role-based login. This user is a potential stakeholder who can review the information made available by staff.

PROPOSED SOLUTION

The proposed solution is a standalone, console-based Passenger Van Rental System developed in Java. It uses a menu to let rental staff select the operation they want to perform. The application organizes its data into related classes and uses text files to retain records between program runs.

The current program provides the following functions:

- Customer management includes adding, viewing, updating, and deleting customer records, subject to the system's rental history restrictions.
- Van management includes adding and deleting vans, viewing vans grouped by status, and searching for a van by plate number.
- Rental processing includes checking customer identity information and van eligibility, recording rental duration and fees, and issuing a simulated email receipt.
- Return processing includes recording returns, calculating late days and applicable late fees, and retaining return and staff details.
- Overdue monitoring includes listing overdue rentals and sending simulated SMS expiration reminders for rentals due today or overdue.
- Maintenance tracking includes starting and finishing maintenance records and viewing maintenance history.
- Input validation and error handling include validating entries such as numbers, contact numbers, email addresses, identification data, van type, and rental duration, as well as displaying handled errors for invalid operations.
- File persistence includes loading and saving vans, customers, rentals, employees, and maintenance records using text files. Notification activity is recorded in a separate log file.

SCOPE AND LIMITATIONS

Scope

- The system manages standard and luxury passenger vans and records each van's plate number, model, seat capacity, availability, and maintenance status.
- It supports customer record creation, viewing, updating, and deletion, with rules to preserve customers linked to rental history.

- It records rental transactions, including rental dates, number of days, total fee, return status, late fee, and staff involved.
- It supports van returns, due-date and overdue checks, and maintenance start/completion records.
- It stores customer identification details and address information used in rental and overdue-rental records.
- It simulates email receipts and SMS reminders and records notification activity in a log file. It does not send messages through a real external email or SMS service.
- It saves and reloads operational records using local text files, including vans, customers, rentals, employees, and maintenance records.
- It provides a menu-driven console interface and validates common user inputs.

Limitations

- The application is a standalone console program. It does not provide a graphical, web, or mobile interface.
- Data is stored in local text files rather than a database management system. The program is intended to run from the folder containing its data files.
- Email receipts and SMS reminders are simulated and displayed on the screen or recorded in a log file. The system does not connect to real messaging services.
- The current program has no login or PIN authentication. Staff IDs are checked for operations that require a staff member, but this is not a full access-control system.
- The program does not provide online booking, online payments, GPS tracking, or integration with external rental platforms.
- The current implementation is not designed for concurrent use by multiple users or devices, and it does not synchronize records across branches or locations.
- The application's reports are limited to the lists and summaries available through its console menu.

JUSTIFICATION FOR USING OBJECT-ORIENTED PROGRAMMING

Object-Oriented Programming (OOP) is suitable for the Passenger Van Rental System because the rental process involves different entities, such as vans, customers, employees, rentals, and maintenance records, each with its own data and behavior. Representing these entities as classes helps organize the application and makes relationships between records easier to manage. The supplied Java code demonstrates several OOP concepts in the following ways:

- **Encapsulation:** Classes such as Van, Customer, Rental, and MaintenanceRecord keep their data in fields and provide methods to access or update that data. This helps keep changes controlled and supports consistent record handling.

- **Inheritance:** StandardVan and LuxuryVan extend the common Van class. Person provides shared person information for related classes such as Customer and Employee.
- **Polymorphism:** The StandardVan and LuxuryVan classes provide their own implementations of van type and fee-related behavior. The system can work with vans through the common Van type while the appropriate subclass behavior is used.
- **Abstraction:** The Notification interface defines a common send operation. EmailNotification and SmsNotification provide different implementations, allowing the rest of the system to request a notification without depending on the details of each implementation.
- **Object relationships:** A Rental links a customer, a van, and the staff involved in the transaction. A MaintenanceRecord links a van and the staff member who recorded its maintenance. These relationships reflect the real rental workflow.
- **Collections:** RentalSystem uses ArrayList collections to manage vans, customers, rentals, employees, and maintenance records, allowing multiple objects of each type to be stored and processed.
- **Exception handling:** RentalException is used to report application-level errors, and the menu loop catches these exceptions to display an error message instead of ending the normal interaction immediately. Input-reading methods also handle invalid numeric input.
- **File persistence:** FileManager reads and writes the system's records in text files. This allows the application to restore data on startup and save changes after menu actions.

These concepts are relevant to the system's actual design rather than being included only as a checklist. They separate responsibilities, support shared behavior, and help the application maintain connected records across rental, return, and maintenance operations.